Ingeniería de soluciones con AI

Semestre 2026-1

Docente:

Francisco Macaya

Noticias: *LangChain Lanza LangSmith Engine.*



Nota: *https://www.langchain.com/blog/introducing-langsmith-engine*

***LangSmith Engine**: Los fallos terminan siendo oportunidades.*

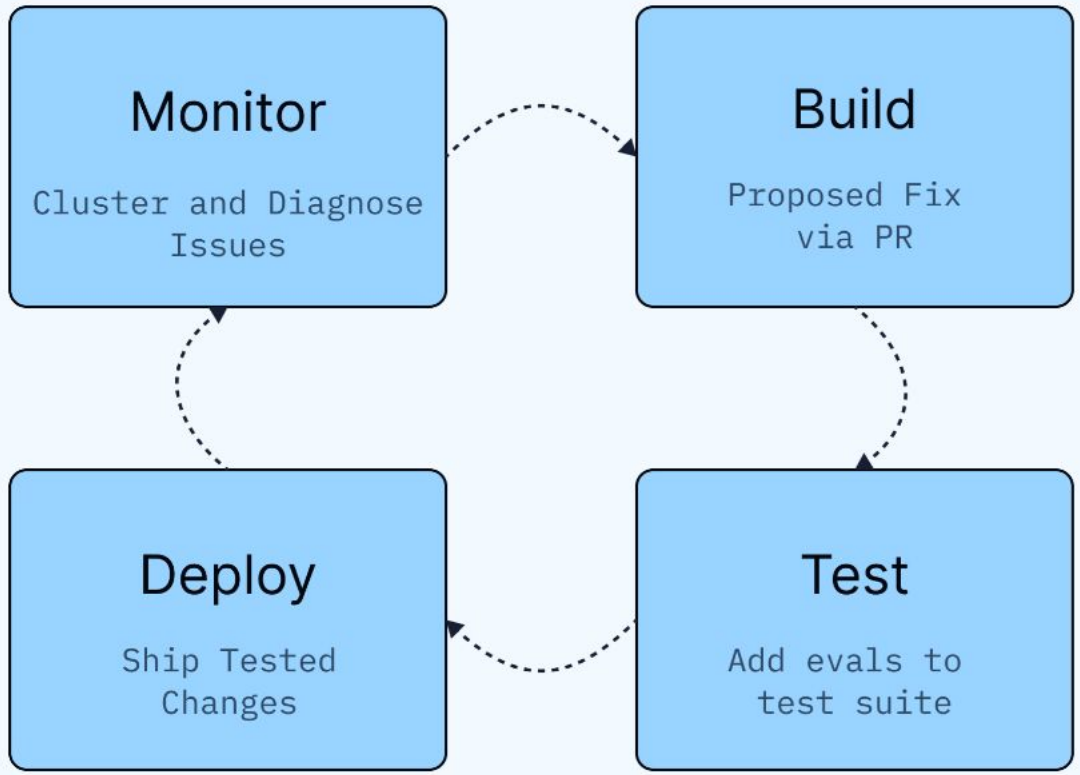
Engine surfaces this as a single named issue, "Agent fails to handle subscription cancellation requests accurately." It shows you the severity (high, affecting 12% of support sessions this week), the timeline (started four days ago, correlating with a recent deployment), and links to the specific traces as evidence.

With your repository connected, Engine reads the relevant code and identifies the root cause. The cancellation tool description is ambiguous, causing the agent to attempt cancellation when users are only asking about their options. Engine drafts a PR with a targeted fix to the tool description.

To keep tracking this behavior going forward, Engine proposes a custom online evaluator scoped to this exact issue, so if the failure pattern recurs after the fix ships, the issue gets resurfaced automatically with updated details.

Engine also pulls the failing traces into a dataset for your offline eval suite, with per-example criteria that define what the correct output should contain. The failures that made it to production become the test cases that keep them out.

That's the full cycle, run autonomously and surfaced for your review. Production signal becomes a clustered issue, then a diagnosed root cause, a proposed fix, and eval coverage.



Nota: *https://www.langchain.com/blog/introducing-langsmith-engine*

¿Que es la Observabilidad?

Observability is the ability to understand the internal state of a system by examining its outputs.

In software, this is typically achieved by analyzing telemetry data such as traces, metrics, and logs.

To make a system observable, it must be instrumented. That is, the code must emit traces, metrics, or logs. The instrumented data must then be sent to an observability backend.

> Observability, on the other hand, makes an assumption stronger than traditional monitoring: that a system's internal states can be inferred from knowledge of its external outputs. When something goes wrong with an observable system, we should be able to figure out what went wrong by looking at the system's logs and metrics without having to ship new code to the system. Observability is about instrumenting your system in a way that ensures that sufficient information about a system's runtime is collected and analyzed so that when something goes wrong, it can help you figure out what goes wrong.

Nota: AI Engineering, Chip Huyen

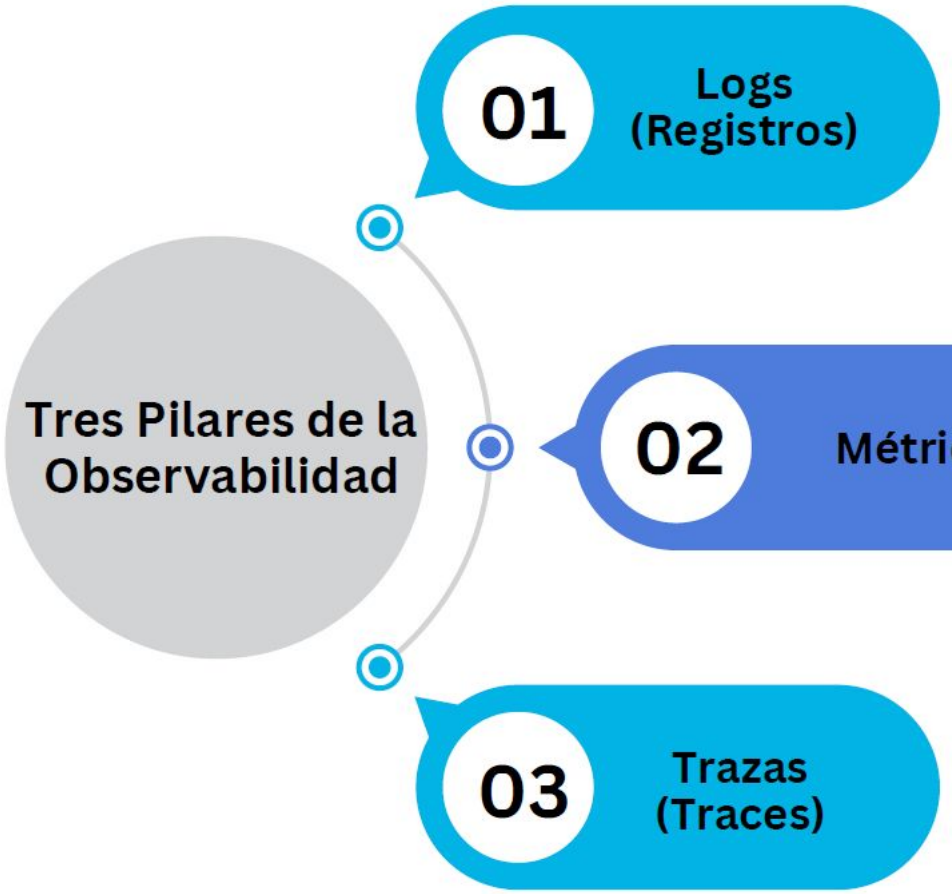
Desde DevOps a GenAI:

- **MTTD (tiempo medio de detección):** cuando ocurre un problema, ¿cuánto se tarda en detectarlo?
- **MTTR (tiempo medio de respuesta):** una vez detectado, ¿cuánto se tarda en resolverlo?
- **CFR (tasa de fallas por cambio):** el porcentaje de cambios o despliegues que provocan fallas y obligan a corregirlos o revertirlos. Si no conoces tu CFR, es momento de rediseñar tu plataforma para hacerla más observable.

Nota: *https://www.langchain.com/blog/introducing-langsmith-engine*

Tres "Stones": ***Logs, Métricas e Traces(Trazas).***

DuocUC



Eventos discretos (prompts, respuestas, uso de herramientas, pensamientos del agente).
Mejores Prácticas: Estructurados, contextuales, centralizados.

Valores numéricos agregados a lo largo del tiempo (latencia, uso de tokens, tasa de éxito).
Mejores Prácticas: Etiquetado, agregación, alertas.

El camino completo de una solicitud a través del sistema, mostrando la secuencia de operaciones.
Mejores Prácticas: Instrumentación, visualización, correlación.

Métricas: *¿Que está saliendo mal o bien?*

Objetivo: El propósito de una métrica es avisarte cuando algo anda mal e identificar oportunidades de mejora.

Antes de definir qué métricas seguir, es importante entender qué tipos de falla quieres detectar y diseñar tus métricas en torno a esas fallas. Por ejemplo, si no quieres que tu **aplicación alucine**, diseña métricas que te ayuden a detectar alucinaciones. **Una métrica útil podría ser si la respuesta de la aplicación se puede inferir del contexto entregado**. Si no quieres que tu aplicación consuma todo tu **crédito de API**, sigue métricas relacionadas con los costos de API, como la **cantidad de tokens de entrada y de salida por solicitud**, o el **costo de tu caché y su tasa de aciertos.**

Si la seguridad es una preocupación, puedes seguir métricas relacionadas con la **toxicidad y detectar información privada y sensible tanto en las entradas como en las salidas**. Mide con qué frecuencia se activan tus barreras de **protección (guardrails)** y con qué frecuencia tu sistema se niega a responder. Detecta también las consultas anómalas a tu sistema, ya que pueden revelar casos límite interesantes o ataques de prompt.

Métricas: *Inferidas a través del lenguaje*

Model quality can also be inferred through user natural language feedback and conversational signals. For example, some easy metrics you can track include the following:

- How often do users stop a generation halfway?
- What's the average number of turns per conversation?
- What's the average number of tokens per input? Are users using your application for more complex tasks, or are they learning to be more concise with their prompts?
- What's the average number of tokens per output? Are some models more verbose than others? Are certain types of queries more likely to result in lengthy answers?
- What's the model's output token distribution? How has it changed over time? Is the model getting more or less diverse?

Nota: AI Engineering, Chip Huyen

Métricas: *North Star*

Given that you'll likely have multiple metrics, it's useful to measure how these metrics correlate to each other and, especially, to your business north star metrics, which can be DAU (daily active user), session duration (the length of time a user spends actively engaged with the application), or subscriptions. Metrics that are strongly correlated to your north star might give you ideas on how to improve your north star. Metrics that are not at all correlated might also give you ideas on what not to optimize for.



Nota: AI Engineering, Chip Huyen

Métricas: *"Click" a métricas de performances.*

DuocUC

Métricas de rendimiento

- **Tiempo hasta el primer token (TTFT):** el tiempo que se demora en generarse el primer token.
- **Tiempo por token de salida (TPOT):** el tiempo que se demora en generarse cada token de salida.
- **Latencia total:** el tiempo total necesario para completar una respuesta.
- **Solicitudes por segundo (RPS):** cuántas solicitudes puede procesar el sistema por segundo.

Costo por interacción = (tokens de prompt × precio por token de entrada) + (tokens de respuesta × precio por token de salida)

- **Tokens de prompt:** todo lo que entra al modelo: el system prompt, el historial de la conversación y el mensaje del usuario.
- **Tokens de respuesta (completion):** los tokens que genera el modelo como respuesta.

Ejercicios:
Construir tu propio dashboard de métricas.

DuocUC

1. **Levanta Grafana:** ejecuta el contenedor y entra a `localhost:3000`.
2. **Levanta la app:** ejecuta `streamlit run app.py` y hazle algunas preguntas.
3. **Piensa en papel:** ¿qué fallas te preocupan? (lentitud, costo, respuestas inventadas, seguridad…). Anota todas las que se te ocurran.
4. **Define una métrica por cada falla:** por ejemplo, TTFT para lentitud o costo por interacción para gasto.
5. **Arma el dashboard:** al final, junta todas tus métricas en un solo dashboard de Grafana.